



Lecture 26

Hash Tables

CS61B, Fall 2025 @ UC Berkeley

Josh Hug and Peyrin Kao

Motivation, Set Implementations

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- ArrayListsSet
- DynamicArrayListsSet
- lowercase strings
- Integer Overflow
- Hash Codes

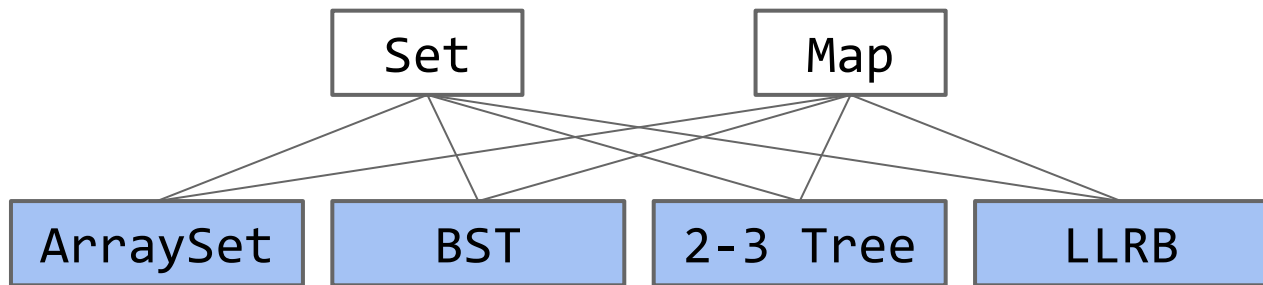
Hash Table Performance and Summary

Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

We've now seen several implementations of the Set (or Map) ADT.



Worst case runtimes

	contains(x)	add(x)	Notes
ArraySet	$\Theta(N)$	$\Theta(N)$	
BST	$\Theta(N)$	$\Theta(N)$	Random trees are $\Theta(\log N)$.
2-3 Tree	$\Theta(\log N)$	$\Theta(\log N)$	Beautiful idea. Very hard to implement.
LLRB	$\Theta(\log N)$	$\Theta(\log N)$	Maintains bijection with 2-3 tree. Hard to implement.

Our search tree based sets require items to be comparable.

- Need to be able to ask “is $X < Y$?” Not true of all types.
- Could we somehow avoid the need for objects to be comparable?

Our search tree sets have excellent performance, but could maybe be better?

- $\Theta(\log N)$ is amazing. 1 billion items is still only height ~ 30 .
- Could we somehow do better than $\Theta(\log N)$?

Today we'll see the answer to both of the questions above is yes.

ArrayOfListsSet

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- **ArrayOfListsSet**
 - DynamicArrayOfListsSet
 - lowercase strings
 - Integer Overflow
 - Hash Codes

Hash Table Performance and
Summary

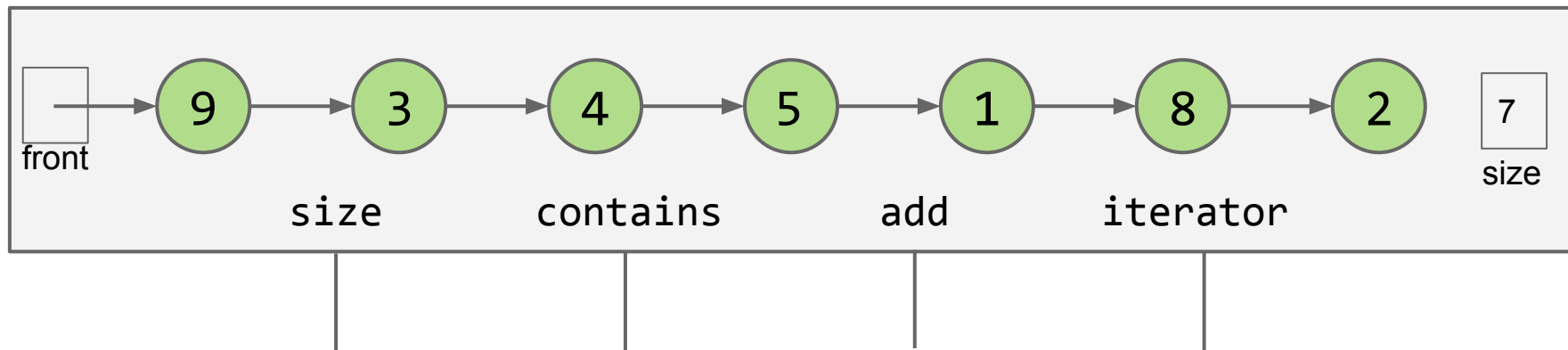
Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

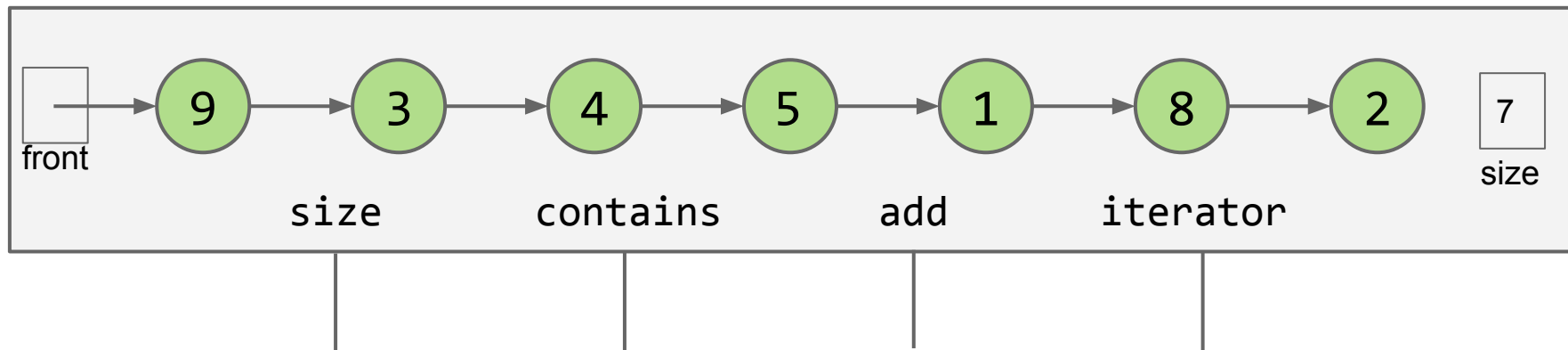
Suppose we have a `LinkedListSet<Integer>`. Let's assume there is no specific order.

- Contains and add are obviously slow.



We considered two different ways we could improve the data structure below.

- Add express lanes (skip lists, out of scope in 61B).
- Transform into a tree like structure (BST and its cousins).



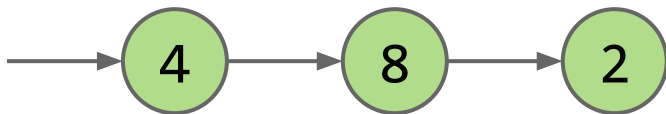
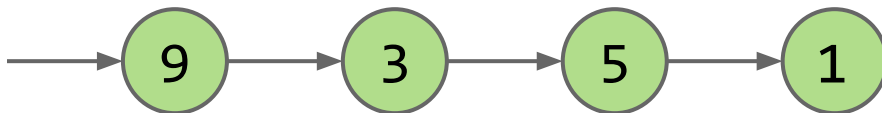
The two approaches above required the data to be in sorted order.

- Our next approach will not.

Key Idea: Multiple Lists

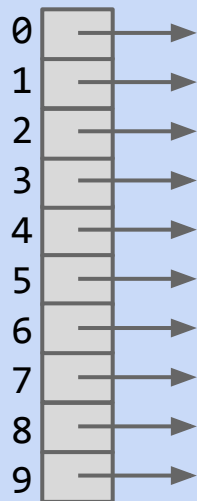
What if we split our list into two lists, one containing the even items, and one containing the odd items?

- add and contains operation will be twice as fast if items are evenly distributed.



Suppose we now have 10 lists.

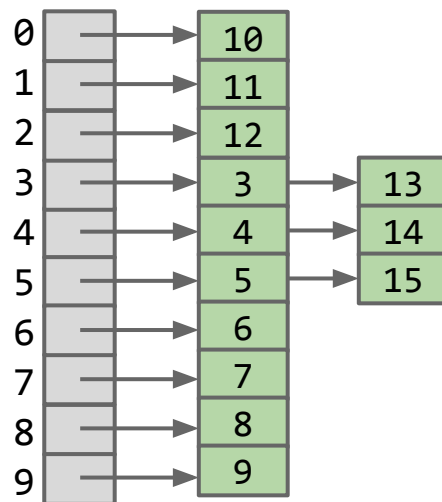
- How do we decide which integers go in which list?
- Example: Suppose we have [3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], where would they go?



Also: How much faster will this new approach be relative to having a single list.

Suppose we now have 10 lists.

- How do we decide which integers go in which list?
- Example: Suppose we have [3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], where would they go? Most natural approach, just % by 10.

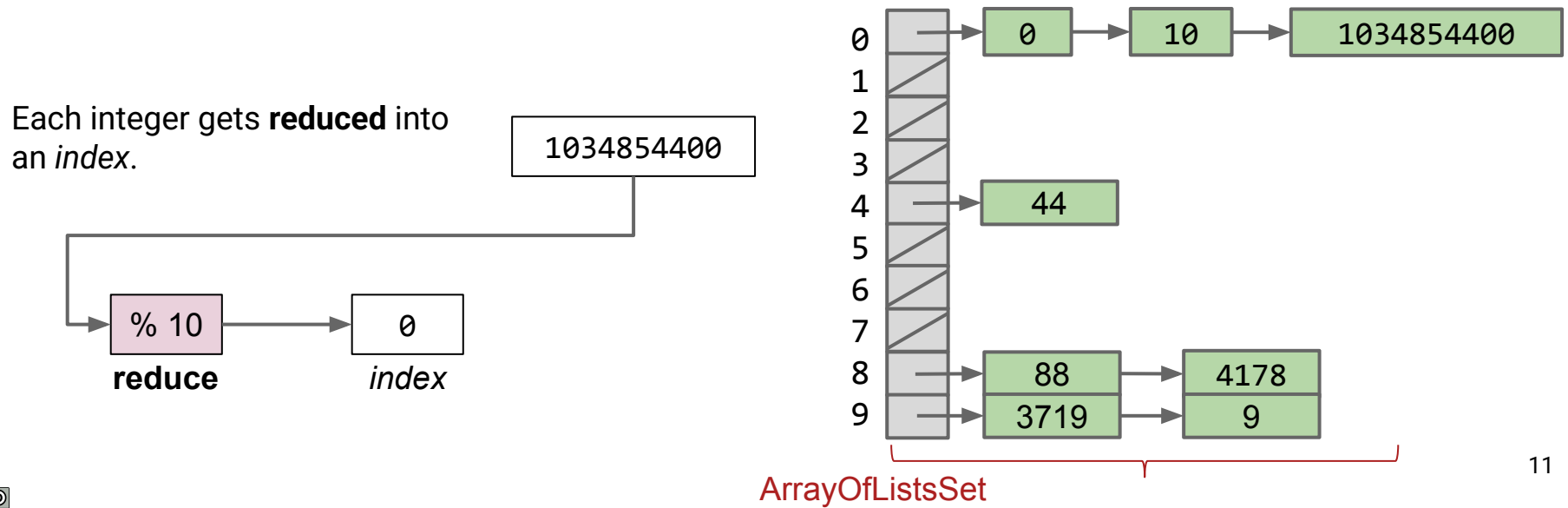


Here: % means modulus.

Below is a picture of our ArrayOfListsSet for $M = 10$.

If we have N items that are evenly distributed, length of each list is N/M .

- Results in an up to M fold speedup.
- If $M = 10$, all operations are up to 10 times faster.

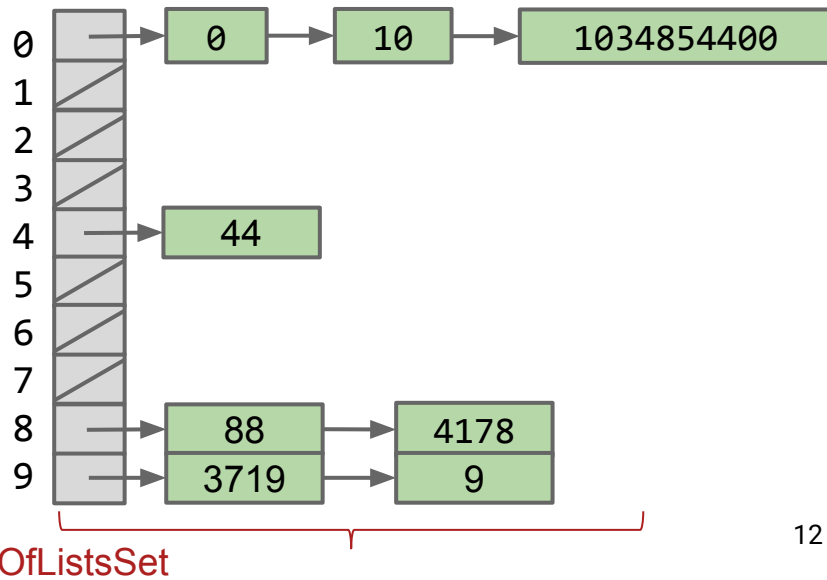
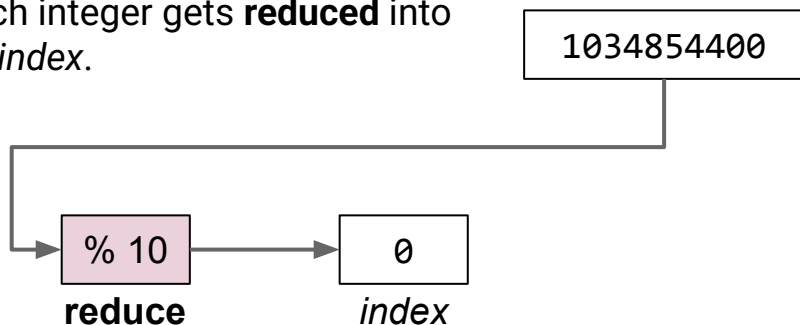


Buckets and Reduction Functions

For this type of data structure, each list is sometimes called a “bucket”.

- May have any number of items in a bucket.
- An integer lands in a bucket based on the reduction function.
 - Other reductions are possible, e.g. number of digits.
 - Modulus is the most natural and best reduction function.

Each integer gets **reduced** into an *index*.



DynamicArrayOfListsSet

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- ArrayOfListsSet
- **DynamicArrayOfListsSet**
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Table Performance and Summary

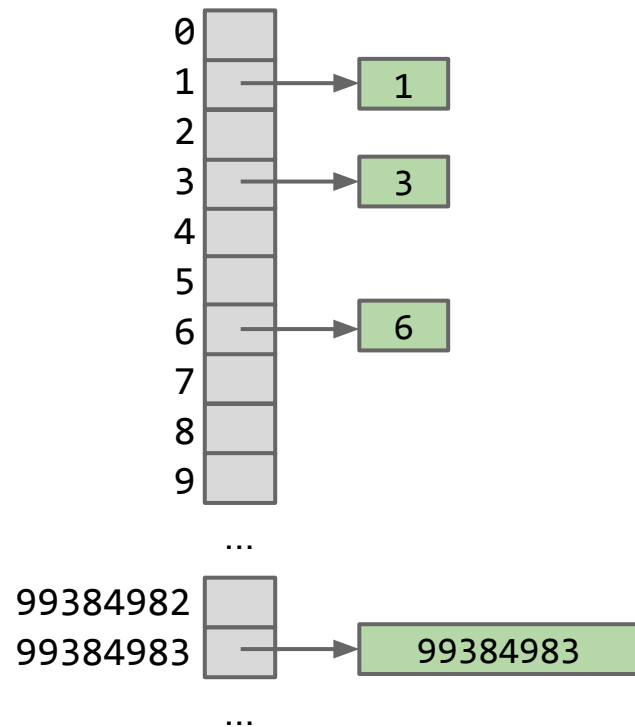
Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

Suppose we have as many lists as there are integers.

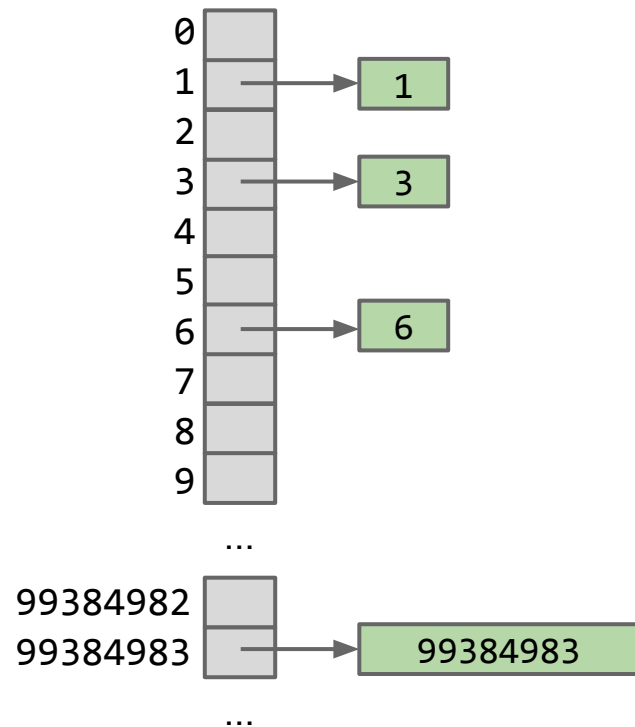
- Every integer gets its own list!
- Runtime is constant for all operations:
 - Contains: Check if that integer's list is empty.
 - Add: If the list is not empty, add the integer.



Issues?

Suppose we have as many lists as there are integers.

- Every integer gets its own list!
- Runtime is constant for all operations:
 - Contains: Check if that integer's list is empty.
 - Add: If the list is not empty, add the integer.



This approach would be wasteful.

- In Java, the smallest integer is -2147483648 and largest integer is 2147483647.
- Thus, we'd need an array of length slightly greater than 4 billion.

We've observed a tradeoff between runtime and space usage:

- For $M = 10$, we got a 10x speedup.
- For $M = \#$ of integers, we get constant time operations.

We've observed a tradeoff between runtime and space usage:

- For $M = 10$, we got a 10x speedup.
- For $M = \#$ of integers, we get constant time operations.

What M should we pick?

We've observed a tradeoff between runtime and space usage:

- For $M = 10$, we got a 10x speedup.
- For $M = \#$ of integers, we get constant time operations.

What M should we pick?

- Could depend on N maybe.
 - If you have more than 10 entries in any single list, then add more buckets and shuffle em appropriately.
 -

We've observed a tradeoff between runtime and space usage:

- For $M = 10$, we got a 10x speedup.
- For $M = \#$ of integers, we get constant time operations.

What M should we pick?

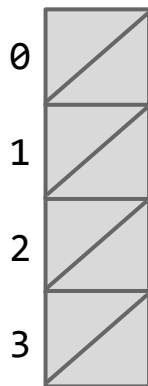
We should start with an M that is small, and let M get larger as N gets larger.

- This is very similar to our resizing arrays (ArrayDeque, ArrayList).

Hash Table Resizing Example

Suppose we set a rule that when N/M is ≥ 1.5 , we double M .

$N = 0$ $M = 4$ $N / M = 0$

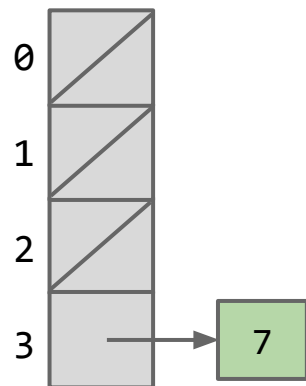


Hash Table Resizing Example

Suppose we set a rule that when N/M is ≥ 1.5 , we double M .

- `add(7)`

$N = 1$ $M = 4$ $N / M = 0.25$

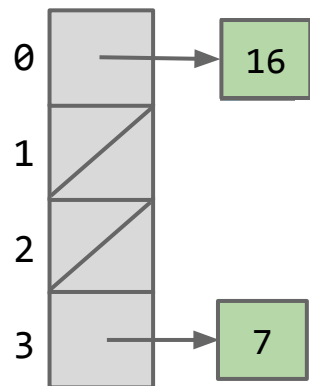


Hash Table Resizing Example

Suppose we set a rule that when N/M is ≥ 1.5 , we double M .

- add(7), add(16)

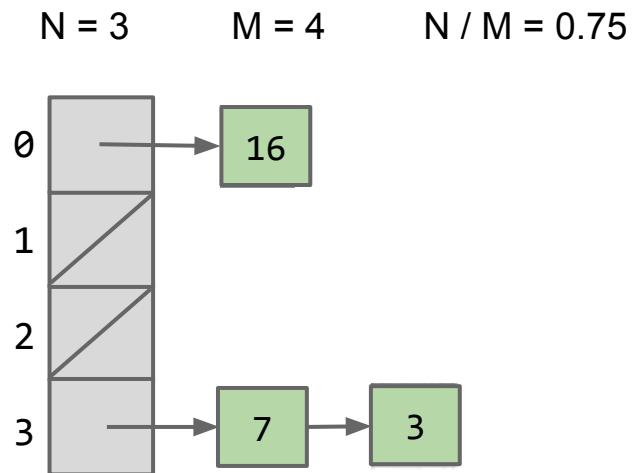
$N = 2$ $M = 4$ $N / M = 0.5$



Hash Table Resizing Example

Suppose we set a rule that when N/M is ≥ 1.5 , we double M .

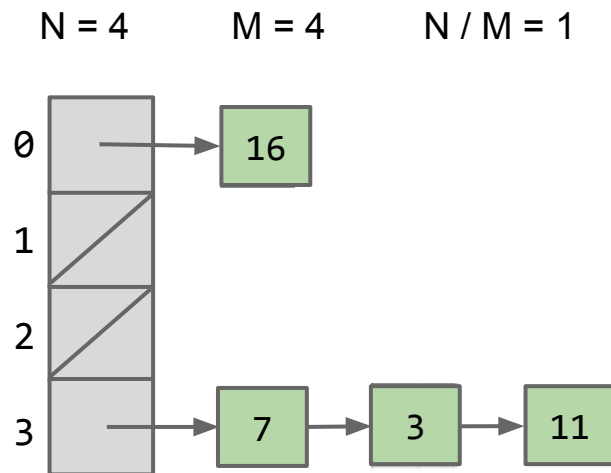
- $\text{add}(7)$, $\text{add}(16)$, $\text{add}(3)$



Hash Table Resizing Example

Suppose we set a rule that when N/M is ≥ 1.5 , we double M .

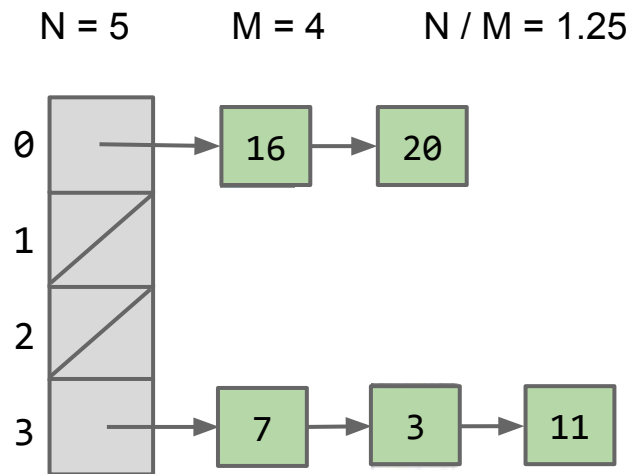
- $\text{add}(7)$, $\text{add}(16)$, $\text{add}(3)$, $\text{add}(11)$



Hash Table Resizing Example

Suppose we set a rule that when N/M is ≥ 1.5 , we double M .

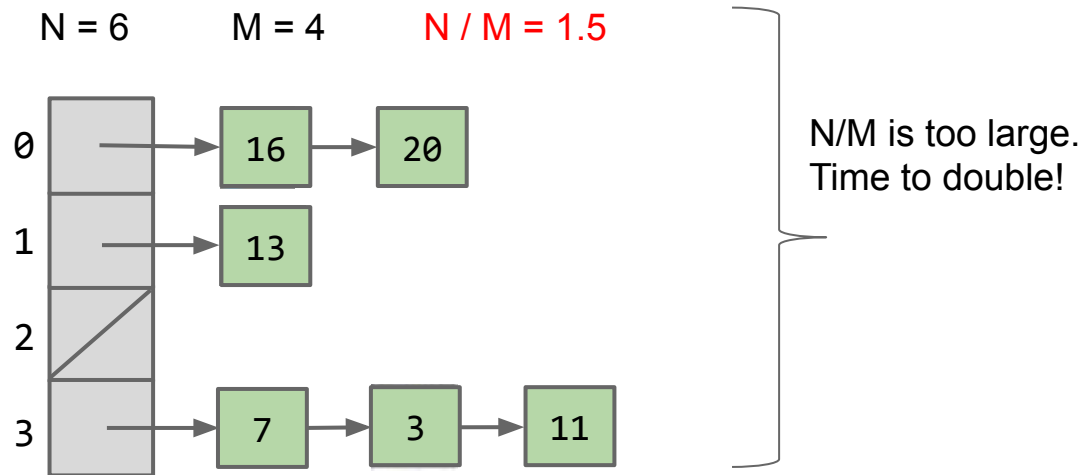
- `add(7)`, `add(16)`, `add(3)`, `add(11)`, `add(20)`



Hash Table Resizing Example

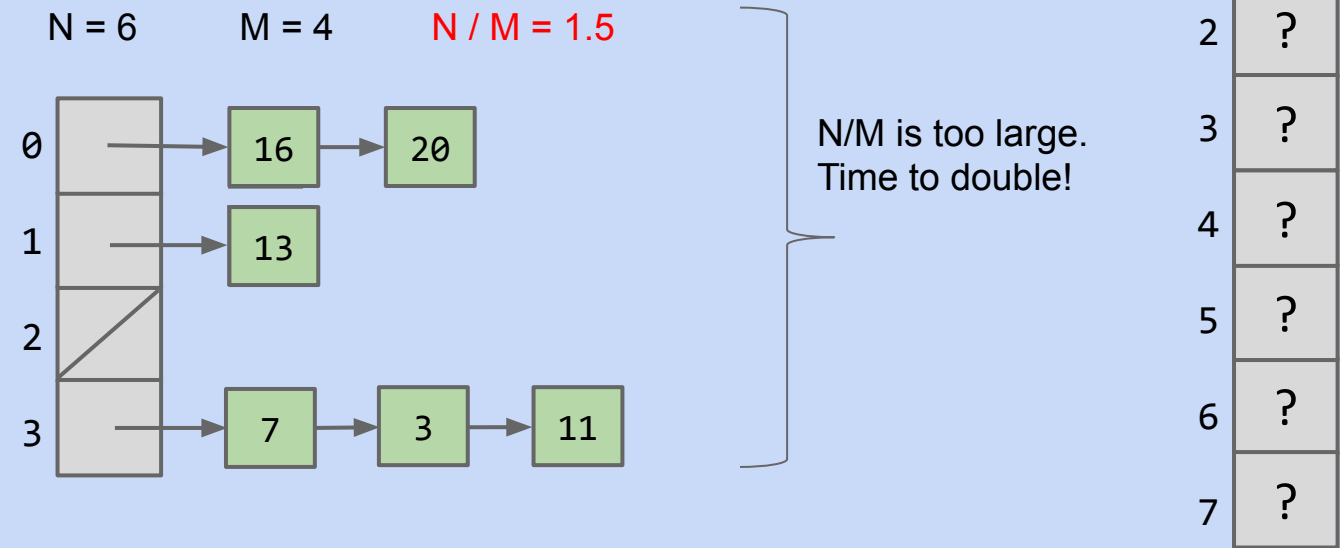
Suppose we set a rule that when N/M is ≥ 1.5 , we double M .

- `add(7)`, `add(16)`, `add(3)`, `add(11)`, `add(20)`, `add(13)`. Resize triggered.



When N/M is ≥ 1.5 , then double M .

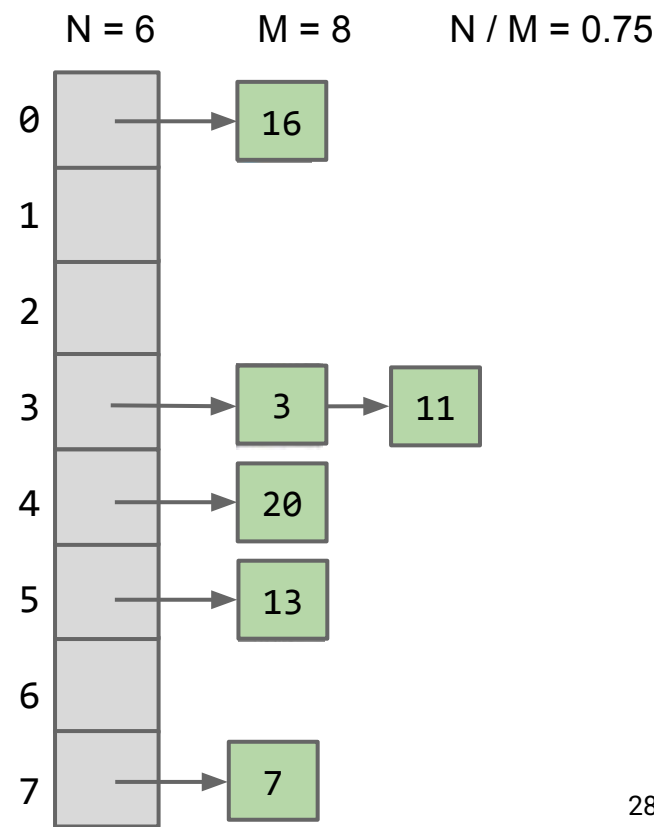
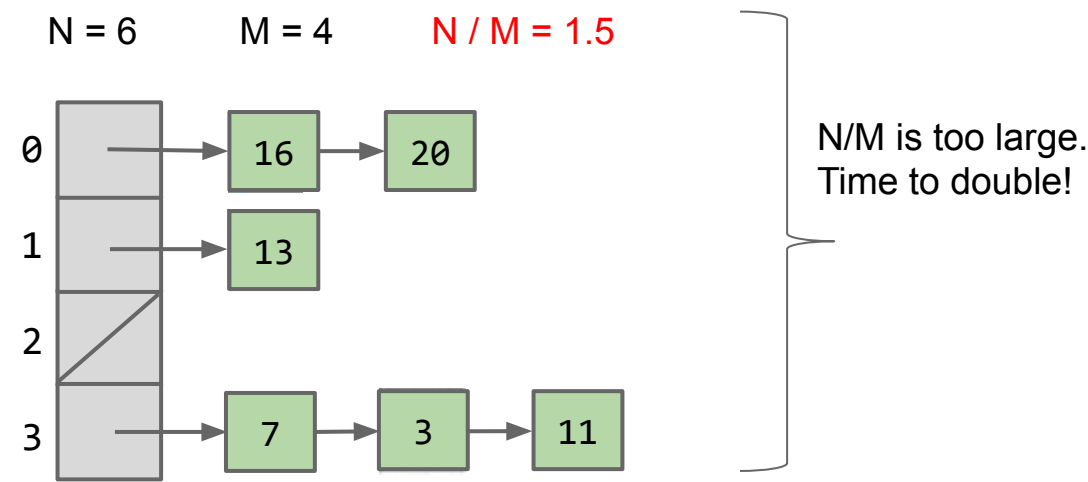
- Yellkey question: Where will the 11 go?
- Or: Draw the results after doubling M .



Hash Table Resizing Example

When N/M is ≥ 1.5 , then double M .

- Draw the results after doubling M .



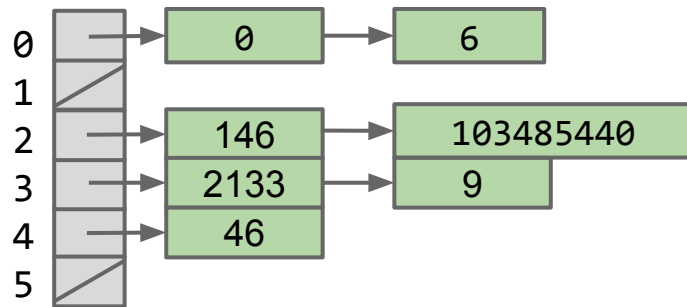
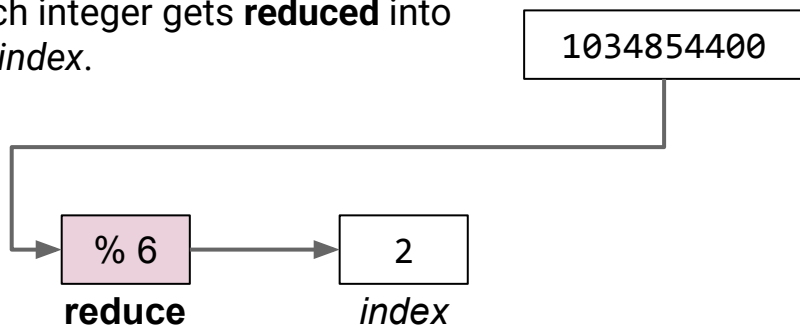
The data structure we just built might be called a DynamicArrayOfListsSet.

- Same as ArrayOfListsSet, but M increases as N increases.

If we have N items that are evenly distributed, length of each list is $\sim N/M$.

- N/M is **constant** asymptotically.
- So operations are constant on average.

Each integer gets **reduced** into an *index*.



We'll think more carefully about runtime later.

lowercase strings

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- `ArrayOfListsSet`
- `DynamicArrayOfListsSet`
- **lowercase strings**
- Integer Overflow
- Hash Codes

Hash Table Performance and
Summary

Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

Goal: Storing Strings

The data structure we have so far is great for storing integers.

- Let's try to figure out how to store Strings.

Suppose we want to add("cat")

The key question:

- What is the **cat**th element of a list?
- One idea: Use the order in the alphabet of the first letter as the list number.
 - **a** = 1, **b** = 2, **c** = 3, ..., **z** = 26

What are some issues with this approach?

Suppose we want to add("cat")

The key question:

- What is the **cat**th element of a list?
- One idea: Use the order in the alphabet of the first letter as the list number.
 - **a** = 1, **b** = 2, **c** = 3, ..., **z** = 26
 - So **cat** would go in bucket 3.

What are some issues with this approach?

- There are some strings that don't start with a letter, e.g. 23yg4iusgd or !iuhf87y63.
- Some letters are more common, e is more common than z.
- Resizing doesn't help because you never use any bucket beyond #26.

Suppose we want to add("cat")

The key question:

- What is the **cat**th element of a list?
- One idea: Use the order in the alphabet of the first letter as the list number.
 - **a** = 1, **b** = 2, **c** = 3, ..., **z** = 26
 - So **cat** would go in bucket 3.

What are some issues with this approach?

- Only the first 26 lists get used.
 - As N grows very large, we end up with a ton of useless lists.
- Can't store "=98yae98fwyawef"

Suppose we want to add("cat")

The key question:

- What is the `cat`th element of a list?

What is another idea? Assume for now we're dealing with only lower case letters in English.

Suppose we want to add("cat")

The key question:

- What is the catth element of a list?

What is another idea? Assume for now we're dealing with only lower case letters in English.

- Length of a string. Works universally, but most buckets won't get used because not many strings are 928347 letters long.
 - Number of vowels in a string works too.
- We're still not using the big buckets if we have thousands of them.
 -

Suppose we want to add("cat")

The key question:

- What is the `cat`th element of a list?

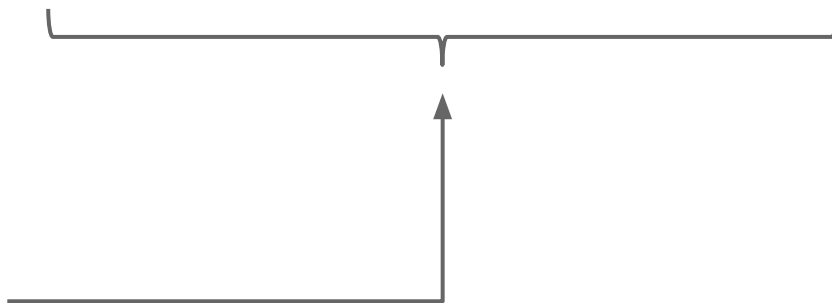
What is another idea? Assume for now we're dealing with only lower case letters in English.

- Treat `cat` as a base 26 number and just use the corresponding bucket.

Treating cat as a Base 26 Number

Use all digits by multiplying each by a power of 26.

- $a = 1, b = 2, c = 3, \dots, z = 26$
- Thus the index of “cat” is $(3 \times 26^2) + (1 \times 26^1) + (20 \times 26^0) = 2074$.



Why this specific pattern?

- Let's review how numbers are represented in decimal.

The Decimal Number System vs. My System for Strings

In the decimal number system, we have 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Want numbers larger than 9? Use a sequence of digits.

Example: 7091 in base 10

$$\bullet \quad 7091_{10} = (7 \times 10^3) + (0 \times 10^2) + (9 \times 10^1) + (1 \times 10^0)$$

Our system for strings is almost the same, but with letters.

- One difference: In decimal numbers, 000 is the same as 0, but with strings aaa is different from a.
- To deal with this, we just don't have a 0 in our system, i.e. a is 1, not 0.

Test Your Understanding (yellkey this)

Convert the word “bee” into a number by using our “powers of 26” strategy.

Reminder: $\text{cat}_{26} = (3 \times 26^2) + (1 \times 26^1) + (20 \times 26^0) = 2074_{10}$

Hint: ‘b’ is letter 2, and ‘e’ is letter 5.

Test Your Understanding

Convert the word “bee” into a number by using our “powers of 26” strategy.

Reminder: $\text{cat}_{26} = (3 \times 26^2) + (1 \times 26^1) + (20 \times 26^0) = 2074_{10}$

Hint: ‘b’ is letter 2, and ‘e’ is letter 5.

- $\text{bee}_{26} = (2 \times 26^2) + (5 \times 26^1) + (5 \times 26^0) = 1487_{10}$

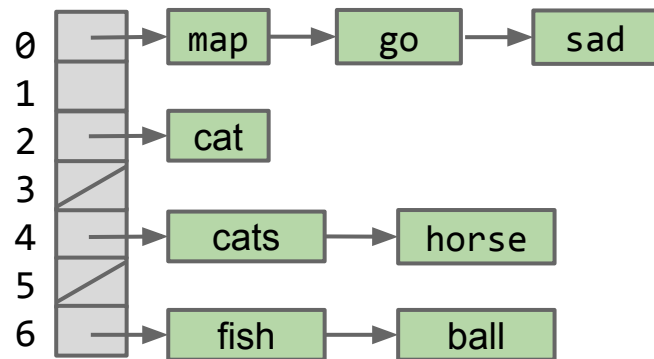
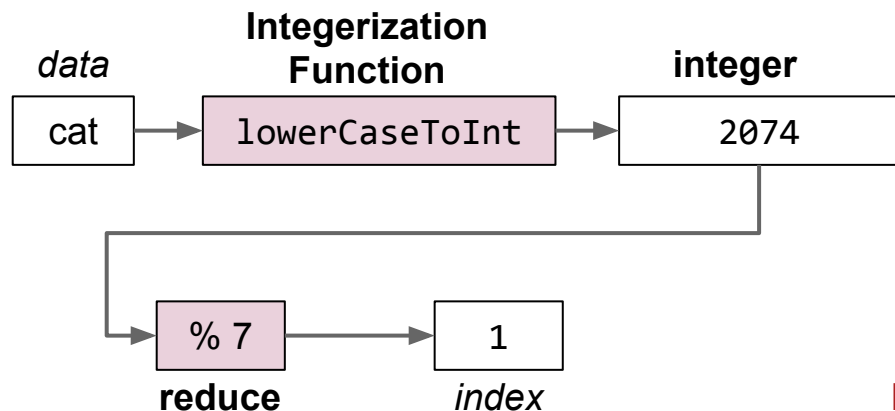
- $\text{cat}_{26} = (3 \times 26^2) + (1 \times 26^1) + (20 \times 26^0) = 2074_{10}$
- $\text{bee}_{26} = (2 \times 26^2) + (5 \times 26^1) + (5 \times 26^0) = 1487_{10}$

As long as we pick a base ≥ 26 , this algorithm is guaranteed to give each lowercase English word a unique number!

- Using base 26, no other words will get the number 1487.

We've now extended our DynamicArrayOfLinkedLists to handle strings.

- *Data* is converted by a **integerization function** into an integer representation.
- The **integer** is then **reduced** to a valid *index*, usually using the modulus operator, e.g. $2348762878 \% 10 = 8$.



Optional exercise: Try to write a function `englishToInt` that can convert English strings to integers by adding characters scaled by powers of 26.

Examples:

- a: 1
- z: 26
- aa: 27
- bee: 1487
- cat: 2074
- dog: ??
- potato: ??

Implementing englishToInt (optional) (solution)

```
/** Converts ith character of String to a letter number.
 * e.g. 'a' -> 1, 'b' -> 2, 'z' -> 26 */
public static int letterNum(String s, int i) {
    int ithChar = s.charAt(i);
    if ((ithChar < 'a') || (ithChar > 'z'))
        { throw new IllegalArgumentException(); }
    return ithChar - 'a' + 1;
}

public static int englishToInt(String s) {
    int intRep = 0;
    for (int i = 0; i < s.length(); i += 1) {
        intRep = intRep * 26;
        intRep = intRep + letterNum(s, i);
    }
    return intRep;
}
```

Integer Overflow

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- `ArrayOfListsSet`
- `DynamicArrayOfListsSet`
- lowercase strings
- **Integer Overflow**
- Hash Codes

Hash Table Performance and
Summary

Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

Using only lowercase English characters is too restrictive.

- What if we want to store strings like “2pac” or “eGg!”?

Suppose we wanted a unique integer for each possible such string.

- Need to assign an integer to all possible characters, e.g. what integer goes with !

Someone has already done this.

- Let's first discuss briefly discuss the ASCII standard.

The most basic character set used by most computers is ASCII format.

- Each possible character is assigned a value between 0 and 127.
- Characters 33 - 126 are “printable”, and are shown below.
- For example, `char c = 'D'` is equivalent to `char c = 68`.

33	!
34	"
35	#
36	\$
37	%
38	&
39	'
40	(
41	)
42	*
43	+
44	,
45	-
46	.
47	/
48	0

49	1
50	2
51	3
52	4
53	5
54	6
55	7
56	8
57	9
58	:
59	;
60	<
61	=
62	>
63	?
64	@

65	A
66	B
67	C
68	D
69	E
70	F
71	G
72	H
73	I
74	J
75	K
76	L
77	M
78	N
79	O
80	P

81	Q
82	R
83	S
84	T
85	U
86	V
87	W
88	X
89	Y
90	Z
91	[
92	\
93	]
94	^
95	_
96	`

97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	i
106	j
107	k
108	l
109	m
110	n
111	o
112	p

113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z
123	{
124	
125	}
126	~

biggest value is 126

Maximum possible value for english-only text including punctuation is 126, so can use 126 as our base in order to ensure unique values for all possible strings.

Examples:

- $\text{bee}_{126} = (98 \times 126^2) + (101 \times 126^1) + (101 \times 126^0) = 1,568,675$
- $\text{2pac}_{126} = (50 \times 126^3) + (112 \times 126^2) + (97 \times 126^1) + (99 \times 126^0)$
 $= 101,809,233$
- $\text{eGg!}_{126} = (98 \times 126^3) + (71 \times 126^2) + (98 \times 126^1) + (33 \times 126^0)$
 $= 203,178,213$

Below is a simple formula which converts a String to an integer.

- Treats String as a base 126 number.

```
public static int asciiToInt(String s) {  
    int intRep = 0;  
    for (int i = 0; i < s.length(); i += 1) {  
        intRep = intRep * 126;  
        intRep = intRep + s.charAt(i);  
    }  
    return intRep;  
}
```

What if we want to use characters beyond ASCII?

chars in Java also support character sets for other languages and symbols.

- `char c = '☂'` is equivalent to `char c = 9730`.
- `char c = '螯'` is equivalent to `char c = 40140`.
- `char c = '헤'` is equivalent to `char c = 54812`.
- This encoding is known as Unicode. Table is too big to list.



Example: Computing Unique Representations of Chinese

The largest possible value for Chinese characters is 40,959*, so we'd need to use this as our base if we want to have a unique representation for all possible strings of Chinese characters.

Example:

- $$\text{守门员}_{40959} = (23432 \times 40959^2) + (38376 \times 40959^1) + (21592 \times 40959^0) = 39,312,024,869,368$$

...	...
守门员	39,3120,2486,9367
守门员	39,3120,2486,9368
守门员	39,3120,2486,9369
...	...

*If you're curious, the last character is: 黎

Hash Codes

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- `ArrayOfListsSet`
- `DynamicArrayOfListsSet`
- lowercase strings
- Integer Overflow
- **Hash Codes**

Hash Table Performance and
Summary

Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

So far, we've tried to map any possible string to a unique integer.

- But in Java, there are only finitely many integers.

That is, we tried to map 守门员 as a base 40959 number, yielding 39,312,024,869,368, but this number doesn't exist in Java as an int.

Note: Other programming languages do not have finitely many integers. Python, for example, allows an integer to take on any value.

- On actual physical computers, some integers will not be able to be stored.

In Java, the largest possible integer is 2,147,483,647.

- If you go over this limit, you overflow, starting back over at the smallest integer, which is -2,147,483,648.
- In other words, the next number after 2,147,483,647 is -2,147,483,648.

```
int x = 2147483647;  
System.out.println(x);  
System.out.println(x + 1);
```

```
jug ~/Dropbox/61b/lec/hashing  
$ javac BiggestPlusOne.java  
$ java BiggestPlusOne  
2147483647  
-2147483648
```

Because Java has a maximum integer, we won't get the numbers we expect!

- With base 126, we will run into overflow even for short strings.
 - Example: $\text{omens}_{126} = 28,196,917,171$, which is much greater than the maximum integer!
 - `asciiToInt('omens')` will give us -1,867,853,901 in Java.

The official term for the number we're computing is “hash code”.

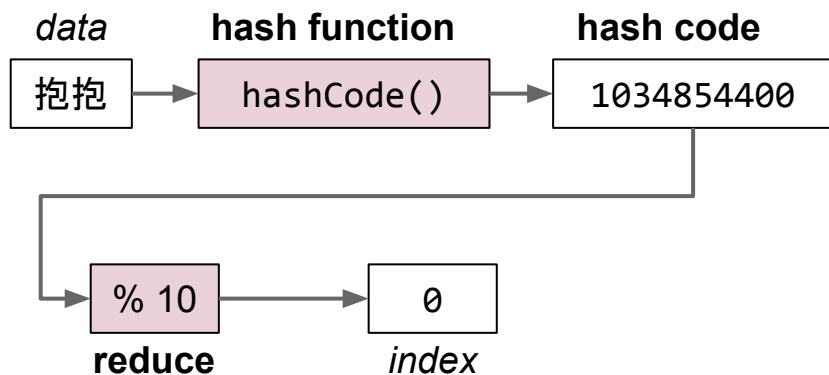
- Via [Wolfram|Alpha](#): a hash code “projects a value from a set with many (or even an infinite number of) members to a value from a set with a fixed number of (fewer) members.”
- Here, our target set is the set of Java integers, which is of size 4,294,967,296.

That is, our integerization function is a “hash code” because the set we're projected onto is fixed.

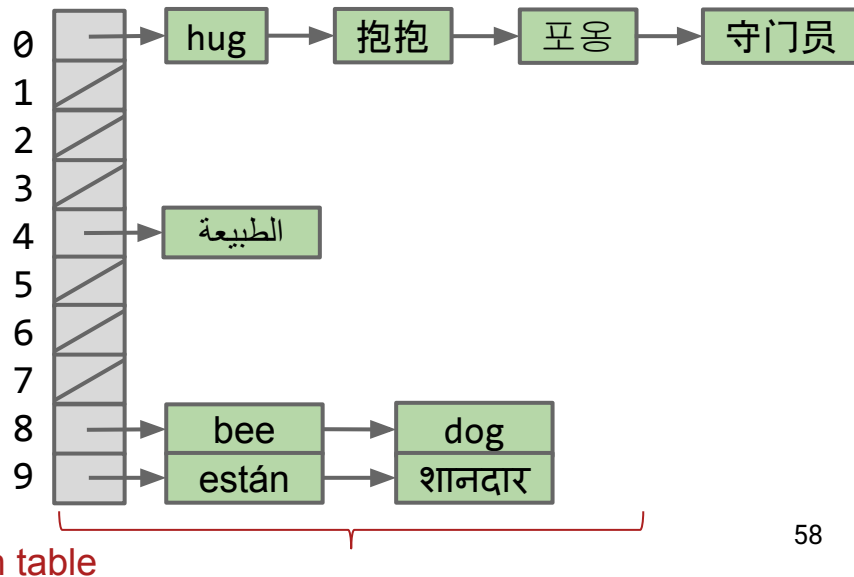
The Hash Table

A `DynamicArrayOfLists` with a finite integerization function is called a **hash table**.

- *Data* is converted by a **hash function** into a finite integer representation called a **hash code**. Java hash codes must be in $[-2,147,483,648$ to $2,147,483,647]$.
- The **hash code** is then **reduced** to a valid *index*, usually using the modulus operator, e.g. $2348762878 \% 10 = 8$.

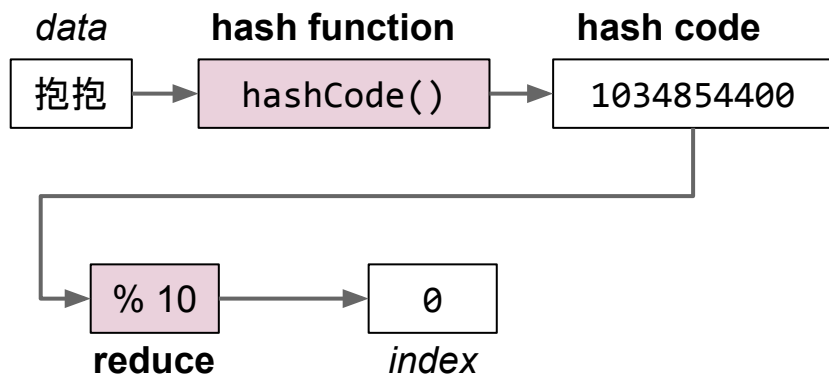


In Java there's a caveat here. Will revisit later.

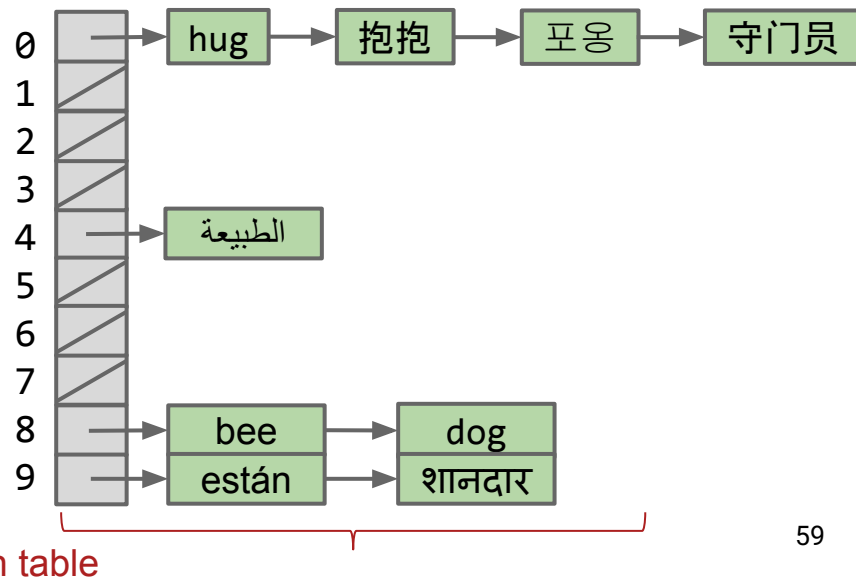


Note, there are other versions of hash tables out there.

- The version we're using is an array of lists.
- This is sometimes called “separate chaining”, where each bucket is a separate chain of items.
- Many more exotic solutions exist (linear probing, cuckoo hashing, using things other than linked lists for buckets, etc).



In Java there's a caveat here. Will revisit later.



Hash Tables in Java

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- `ArrayOfListsSet`
- `DynamicArrayOfListsSet`
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Table Performance and Summary

Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

Hash tables are the most popular implementation for sets and maps.

- Great performance in practice.
- Don't require items to be comparable.
- Implementations often relatively simple.
- Python dictionaries are just hash tables in disguise.

In Java, implemented as `java.util.HashMap` and `java.util.HashSet`.

- How does a `HashMap` know how to compute each object's hash code?
 - Good news: It's not "implements `Hashable`".
 - Instead, all objects in Java must implement a `.hashCode()` method.

All classes are hyponyms of Object.

- `String toString()`
- **`boolean equals(Object obj)`**
- `int hashCode()` ←
- `Class<?> getClass()`
- `protected Object clone()`
- `protected void finalize()`
- `void notify()`
- `void notifyAll()`
- `void wait()`
- `void wait(long timeout)`
- `void wait(long timeout, int nanos)`

From earlier in class.

This is where Java implements hash codes.

Won't discuss or use in 61B.

Default implementation of hashCode returns memory address.

Java's actual hashCode function for Strings below (code cleaned up slightly):

- “守门员” and “拾囍囍” map to 23,729,400.

```
public int hashCode(String s) {  
    int intRep = 0;  
    for (int i = 0; i < s.length(); i += 1) {  
        intRep = intRep * 31;  
        intRep = intRep + s.charAt(i);  
    }  
    return intRep;  
}
```

Due to overflow, multiple strings have the same hashcode, e.g. the two calls below both return 23,729,400.

- “守门员”.hashCode()
- “拾囍囍”.hashCode()

More examples of Real Java HashCodes for Strings

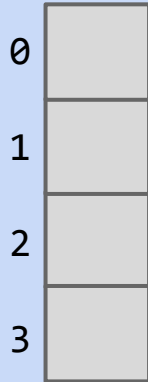
```
System.out.println("a".hashCode());  
System.out.println("bee".hashCode());  
System.out.println("포옹".hashCode());  
System.out.println("kamala lifefully".hashCode());  
System.out.println("đậu hũ".hashCode());
```

```
jug ~/Dropbox/61b/lec/hashing  
$ java JavaHashCodeExamples  
"a" 97  
"bee" 97410  
"포옹" 1732557  
"kamala lifefully" 1732557  
"đậu hũ" -2108180664
```




Suppose that 's hash code is -1.

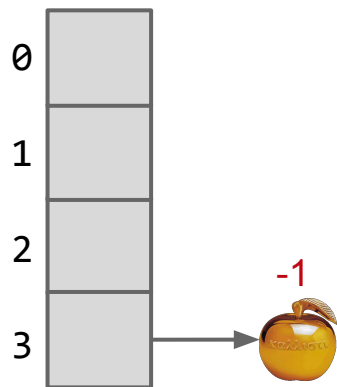
- Philosophically, into which bucket is it most natural to place this item?





Suppose that 's hash code is -1.

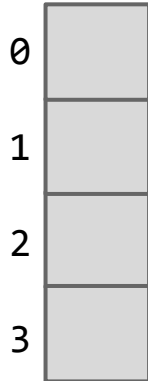
- Philosophically, into which bucket is it most natural to place this item?
 - I say 3, since $-1 \rightarrow 3$, $0 \rightarrow 0$, $1 \rightarrow 1$, $2 \rightarrow 2$, $3 \rightarrow 3$, $4 \rightarrow 0$, ...





Suppose that 's hash code is -1.

- Unfortunately, $-1 \% 4 = -1$. Will result in index errors!
- Use `Math.floorMod` instead.

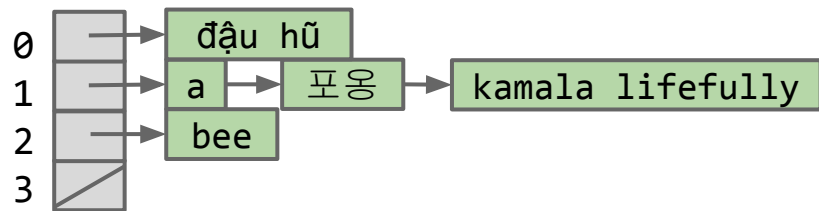
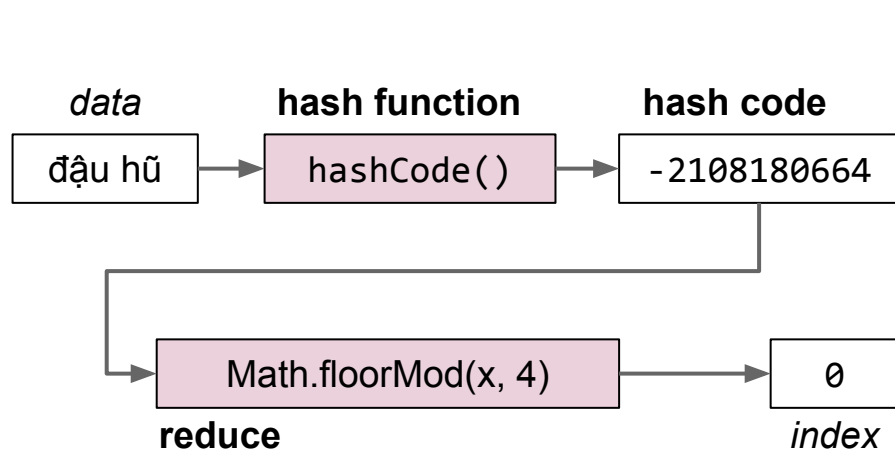


```
public class ModTest {  
    public static void main(String[] args) {  
        System.out.println(-1 % 4);  
        System.out.println(Math.floorMod(-1, 4));  
    }  
}
```

```
$ java ModTest  
-1  
3
```

Java hash tables:

- *Data* is converted by the **hashCode** method an integer representation called a **hash code**.
- The **hash code** is then **reduced** to a valid *index*, using something like the floorMod function, e.g. $\text{Math.floorMod}(1732557 \% 4) = 8$.



Warning #1: Never store objects that can change in a HashSet or HashMap!

- Such objects are also called “mutable” objects, e.g. they can change.
 - Example: You’d never want to make a `HashSet<List<Integer>>`.
- If an object’s variables changes, then its `hashCode` changes. May result in items getting lost.

Warning #2: Never override `equals` without also overriding `hashCode`.

- Can also lead to items getting lost and generally weird behavior.
- HashMaps and HashSets use `equals` to determine if an item exists in a particular bucket.

We’ll come back to these warnings later.

Hash Table Performance and Summary

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- `ArrayOfListsSet`
- `DynamicArrayOfListsSet`
- lowercase strings
- Integer Overflow
- Hash Codes

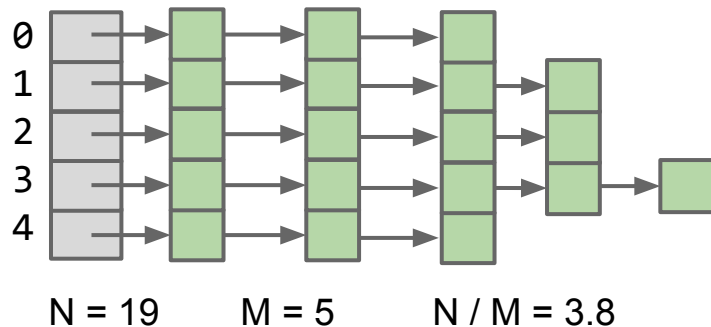
Hash Table Performance and Summary

Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

Hash Table Runtime with No Resizing



Suppose we have:

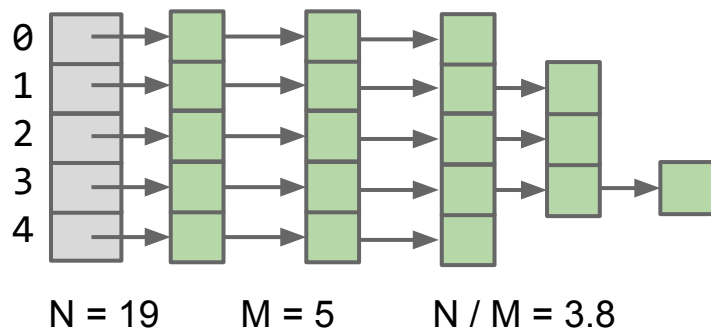
- An fixed number of buckets M .
- An increasing number of items N .

Average list is around N/M items

Even if items are spread out evenly, lists are of length $Q = N/M$.

- For $M = 5$, that means $Q = \Theta(N)$. Results in linear time operations.

Resizing Hash Table Runtime



Suppose we have:

- An increasing number of buckets M .
- An increasing number of items N .

As long as $M = \Theta(N)$, then $O(N/M) = O(1)$.

Assuming items are evenly distributed (as above), lists will be approximately N/M items long, resulting in $\Theta(N/M)$ runtimes.

- By doubling every time N gets too big, we ensure that $N/M = O(1)$.
- Thus, worst case runtime for all operations is $\Theta(N/M) = \Theta(1)$.
 - ... unless that operation causes a resize.
 - ... and again, we're assuming even distribution of items.

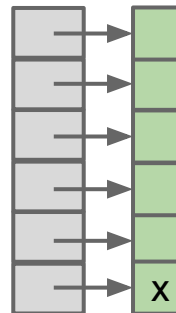
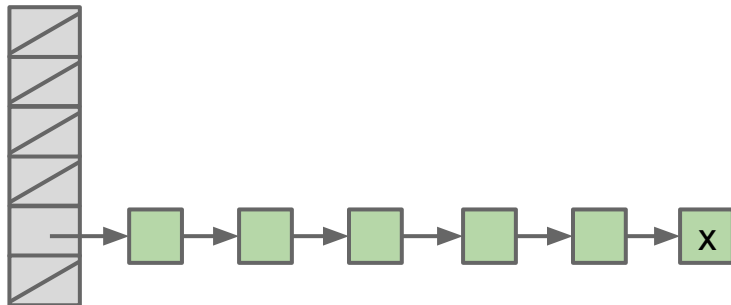
Regarding Even Distribution

Even distribution of item is critical for good hash table performance.

- Both tables below have load factor of $N/M = 1$.
- Left table is much worse!
 - Contains is $\Theta(N)$ for x .

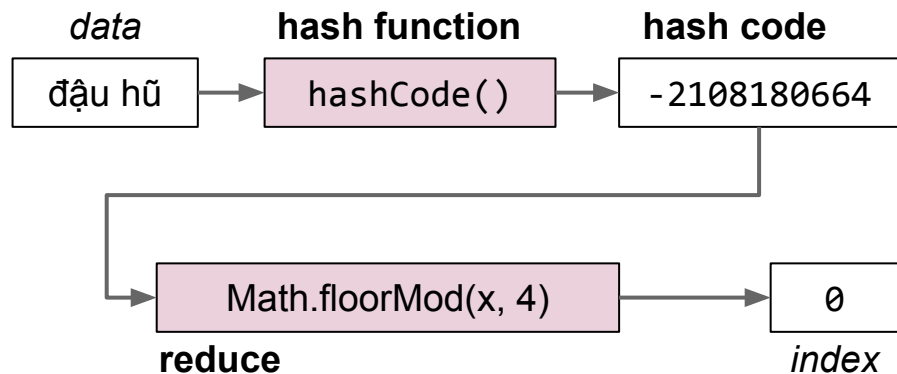
Will need to discuss how to ensure even distribution.

- See extra video and slides for more on ensuring an even distribution.



Hash tables:

- *Data* is converted into a hash code.
- The **hash code** is then **reduced** to a valid *index*.
- *Data* is then stored in a bucket corresponding to that *index*.
- Resize when load factor N/M exceeds some constant.
- If items are spread out nicely, you get $\Theta(1)$ average runtime.



	contains(x)	add(x)
Bushy BSTs	$\Theta(\log N)$	$\Theta(\log N)$
Separate Chaining Hash Table With No Resizing	$\Theta(N)$	$\Theta(N)$
... With Resizing	$\Theta(1)^\dagger$	$\Theta(1)^{*\dagger}$

*: Indicates “on average”.

†: Assuming items are evenly spread.

Creating a Good Hash Code (extra)

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- `ArrayOfListsSet`
- `DynamicArrayOfListsSet`
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Table Performance and Summary

Hash Tables in Java

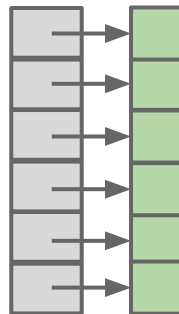
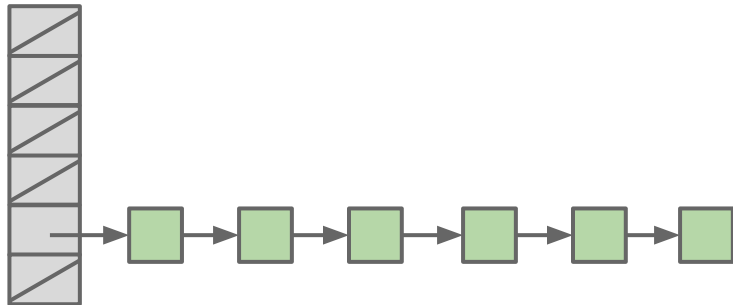
Creating a Good Hash Code (extra)

Linear Probing (extra)

What Makes a good .hashCode()?

Goal: We want hash tables that look like the table on the right.

- Want a hashCode that spreads things out nicely on real data.
 - Example #1: return 0 is a bad hashCode function.
 - Example #2: just returning the first character of a word, e.g. “cat” → 3 was also a bad hash function.
 - Example #3: Adding chars together is bad. “ab” collides with “ba”.
 - Example #4: returning string treated as a base B number can be good.
- Writing a good hashCode() method **can be tricky**.



How do you make hashbrowns?

- Chopping a potato into nice predictable segments? No way!
- Similarly, adding up the characters is not nearly “random” enough.

Can think of multiplying data by powers of some base as ensuring that all the data gets scrambled together into a seemingly random integer.



Example hashCode Function

The Java 8 hash code for strings. Two major differences from our hash codes:

- Represents strings as a base 31 number.
 - Why such a small base? Real hash codes don't care about uniqueness.
- Stores (caches) calculated hash code so future `hashCode` calls are faster.

```
@Override
public int hashCode() {
    int h = cachedHashValue;
    if (h == 0 && this.length() > 0) {
        for (int i = 0; i < this.length(); i++) {
            h = 31 * h + this.charAt(i);
        }
        cachedHashValue = h;
    }
    return h;
}
```

Java's `hashCode()` function for Strings:

- $$h(s) = s_0 \times 31^{n-1} + s_1 \times 31^{n-2} + \dots + s_{n-1}$$

Our `asciiToInt` function for Strings:

- $$h(s) = s_0 \times 126^{n-1} + s_1 \times 126^{n-2} + \dots + s_{n-1}$$

Which is better?

- Might seem like 126 is better. Ignoring overflow, this ensures a unique numerical representation for all ASCII strings.
- ... but overflow is a particularly bad problem for base 126!

Major collision problem:

- “geocronite is the best thing on the earth.”.hashCode() yields 634199182.
- “flan is the best thing on the earth.”.hashCode() yields 634199182.
- “treachery is the best thing on the earth.”.hashCode() yields 634199182.
- “Brazil is the best thing on the earth.”.hashCode() yields 634199182.

Any string that ends in the same last 32 characters has the same hash code.

- Why? Because of overflow.
- Basic issue is that $126^{32} = 126^{33} = 126^{34} = \dots 0$.
 - Thus upper characters are all multiplied by zero.
 - See CS61C for more.

A typical hash code base is a small prime.

- Why prime?
 - Never even: Avoids the overflow issue on previous slide.
 - Lower chance of resulting hashCode having a bad relationship with the number of buckets: See study guide problems and hw3.
- Why small?
 - Lower cost to compute.

A full treatment of good hash codes is well beyond the scope of our class.

How do you make hashbrowns?

- Chopping a potato into nice predictable segments? No way!

Using a prime base yields better “randomness” than using something like base 126.



Example: Hashing a Collection

Lists are a lot like strings: Collection of items each with its own hashCode:

```
@Override
public int hashCode() {
    int hashCode = 1;
    for (Object o : this) {
        hashCode = hashCode * 31;
        hashCode = hashCode + o.hashCode();
    }
    return hashCode;
}
```

elevate/smear the current hash code

add new item's hash code

To save time hashing: Look at only first few items.

- Higher chance of collisions but things will still work.

Example: Hashing a Recursive Data Structure

Computation of the hashCode of a recursive data structure involves recursive computation.

- For example, binary tree hashCode (assuming sentinel leaves):

```
@Override
public int hashCode() {
    if (this.value == null) {
        return 0;
    }
    return this.value.hashCode() +
        31 * this.left.hashCode() +
        31 * 31 * this.right.hashCode();
}
```

Linear Probing (extra)

Lecture 26, CS61B, Fall 2025

Motivation, Set Implementations

Deriving Hash Tables

- `ArrayOfListsSet`
- `DynamicArrayOfListsSet`
- lowercase strings
- Integer Overflow
- Hash Codes

Hash Table Performance and
Summary

Hash Tables in Java

Creating a Good Hash Code (extra)

Linear Probing (extra)

Instead of using linked lists, an alternate and more exotic strategy is “open addressing”.

- Set is stored as an array of items. Index tells you where to put the item.

If target location is already occupied, use a different location, e.g.

- Linear probing: Use next address, and if already occupied, just keep scanning one by one.
 - Demo: <http://goo.gl/o5EDvb>
- Quadratic probing: Use next address, and if already occupied, try looking 4 ahead, then 9 ahead, then 16 ahead, ...
- Many more possibilities. See the optional reading for today (or CS170) for a more detailed look.

In 61B, we'll use the “separate chaining” approach, where we have linked lists.

Citations

<http://www.nydailynews.com/news/national/couple-calls-911-forgotten-mcdonalds-hash-browns-article-1.1543096>

http://en.wikipedia.org/wiki/Pigeonhole_principle#mediaviewer/File:TooManyPigeons.jpg

https://cookingplanit.com/public/uploads/inventory/hashbrown_1366322674.jpg

What is the distinction between hash set, hash map, and hash table?

A hash set is an implementation of the Set ADT using the “hash table” as its engine.

A hash map is an implementation of the Map ADT using the “hash table” as its engine.

A “hash table” is a way of storing information, where you have M buckets that store N items. Each item has a “hashCode” that tells you which of M buckets to put that item in.

<https://www.yellkey.com/iterate>

